Walchand College of Engineering, Sangli

Department of Computer Science & Engineering

Computer Algorithm Laboratory
Assignment No. 10 — Complexity Classes and Approximation Algorithms

Name : Atharv V Kulkarni

PRN : 245109074

Class : TY B.Tech CSE (B)

Q1. N-Queens Problem (Backtracking)

Problem Statement:

In defense systems or sensor networks, placing radars/detectors such that none interfere with each other mimics the n-Queen problem. Implement the n x n Queen problem using backtracking, extend the solution for n = 8 to display all possible solutions, and analyze the time complexity while discussing why it is an NP problem.

Approach:

approach_1: place queens row by row. for the current row, try every column; before placing, check isSafe against all previously placed queens in the same column and along both diagonals (|cols[r]-col| == |r-row| catches both diagonals at once). if safe, place the queen and recurse to the next row; if the recursive call fails to complete the board, undo the placement (backtrack) and try the next column. when row == n, a full valid placement has been found and is stored.

Time and Space Complexity:

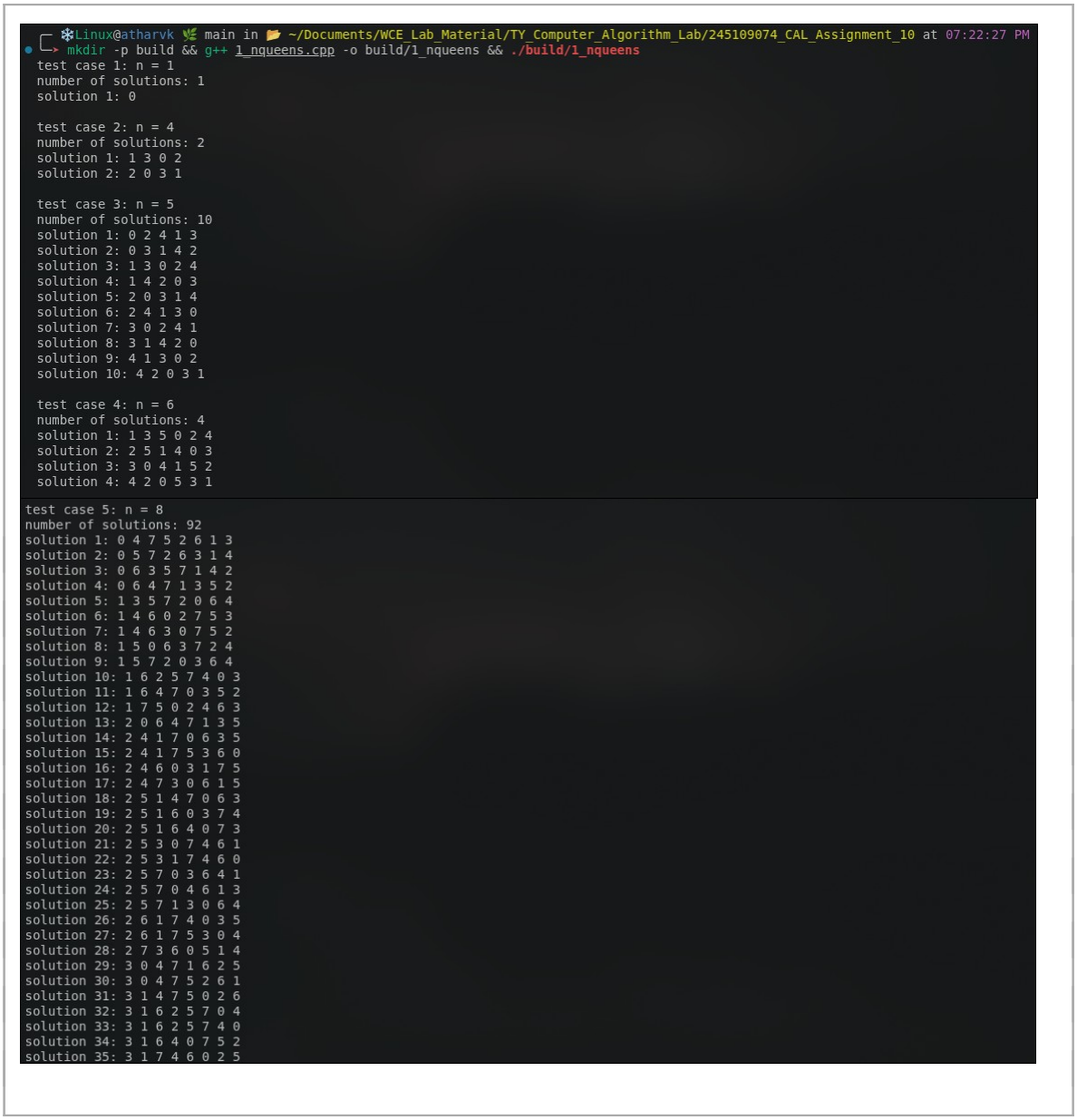
time complexity: O(n!) worst case for the backtracking search (with constraint propagation making the practical runtime far smaller), O(n^2) to verify any single candidate solution.

space complexity: O(n) for the recursion depth and the per-row column array, plus O(number of solutions x n) to store all solutions found.

Code:

```
#include<iostream>
#include<vector>
using namespace std;
bool isSafe(vector<int>&cols,int row,int col){
   for(int r=0;r<row;r++){
      if(cols[r]==col)
         return false;
      if(abs(cols[r]-col)==abs(r-row))
         return false;
   }
   return true;
}
void solve(int n,int row,vector<int>&cols,vector<vector<int>>&solutions){
   if(row==n){
      solutions.push_back(cols);
      return;
   }
   for(int col=0;col<n;col++){
      if(isSafe(cols,row,col)){
         cols[row]=col;
         solve(n,row+1,cols,solutions);
      }
   }
}
void runTestCase(int tc,int n){
   vector<int>cols(n,-1);
   vector<vector<int>>solutions;
   solve(n,0,cols,solutions);
   cout<<"test case "<<tc<<": n = "<<n<<endl;
   cout<<"number of solutions: "<<solutions.size()<<endl;
   for(int i=0;i<(int)solutions.size();i++){
      cout<<"solution "<<i+1<<": ";
      for(int c:solutions[i])
         cout<<c<<" ";
      cout<<endl;
   }
   cout<<endl;
}
int main(){
   runTestCase(1,1);
   runTestCase(2,4);
   runTestCase(3,5);
   runTestCase(4,6);
   runTestCase(5,8);
   return 0;
}
```

Output:


Linux@atharvk main in ~/Documents/WCE_Lab_Material/TY_Computer_Algorithm_Lab/245109074_CAL_Assignment_10 at 07:22:27 PM
mkdir -p build && g++ 1_nqueens.cpp -o build/1_nqueens && ./build/1_nqueens
test case 1: n = 1
number of solutions: 1
solution 1: 0
test case 2: n = 4
number of solutions: 2
solution 1: 1 3 0 2
solution 2: 2 0 3 1
test case 3: n = 5
number of solutions: 10
solution 1: 0 2 4 1 3
solution 2: 0 3 1 4 2
solution 3: 1 3 0 2 4
solution 4: 1 4 2 0 3
solution 5: 2 0 3 1 4
solution 6: 2 4 1 3 0
solution 7: 3 0 2 4 1
solution 8: 3 1 4 2 0
solution 9: 4 1 3 0 2
solution 10: 4 2 0 3 1
test case 4: n = 6
number of solutions: 4
solution 1: 1 3 5 0 2 4
solution 2: 2 5 1 4 0 3
solution 3: 3 0 4 1 5 2
solution 4: 4 2 0 5 3 1
test case 5: n = 8
number of solutions: 92
solution 1: 0 4 7 5 2 6 1 3
solution 2: 0 5 7 2 6 3 1 4
solution 3: 0 6 3 5 7 1 4 2
solution 4: 0 6 4 7 1 3 5 2
solution 5: 1 3 5 7 2 0 6 4
solution 6: 1 4 6 0 2 7 5 3
solution 7: 1 4 6 3 0 7 5 2
solution 8: 1 5 0 6 3 7 2 4
solution 9: 1 5 7 2 0 3 6 4
solution 10: 1 6 2 5 7 4 0 3
solution 11: 1 6 4 7 0 3 5 2
solution 12: 1 7 5 0 2 4 6 3
solution 13: 2 0 6 4 7 1 3 5
solution 14: 2 4 1 7 0 6 3 5
solution 15: 2 4 1 7 5 3 6 0
solution 16: 2 4 6 0 3 1 7 5
solution 17: 2 4 7 3 0 6 1 5
solution 18: 2 5 1 4 7 0 6 3
solution 19: 2 5 1 6 0 3 7 4
solution 20: 2 5 1 6 4 0 7 3
solution 21: 2 5 3 0 7 4 6 1
solution 22: 2 5 3 1 7 4 6 0
solution 23: 2 5 7 0 3 6 4 1
solution 24: 2 5 7 0 4 6 1 3
solution 25: 2 5 7 1 3 0 6 4
solution 26: 2 6 1 7 4 0 3 5
solution 27: 2 6 1 7 5 3 0 4
solution 28: 2 7 3 6 0 5 1 4
solution 29: 3 0 4 7 1 6 2 5
solution 30: 3 0 4 7 5 2 6 1
solution 31: 3 1 4 7 5 0 2 6
solution 32: 3 1 6 2 5 7 0 4
solution 33: 3 1 6 2 5 7 4 0
solution 34: 3 1 6 4 0 7 5 2
solution 35: 3 1 7 4 6 0 2 5

Q2. Graph Colouring (Backtracking)

Problem Statement:

In wireless networks or exam scheduling, we must assign frequencies/time slots so that no two adjacent nodes (or exams) share the same colour. Implement graph colouring using backtracking given a graph adjacency matrix, output the minimum number of colours required along with the colouring assignment, and verify that adjacent vertices do not share the same colour.

Approach:

approach_1: for a candidate number of colours m, try to colour the graph vertex by vertex using backtracking. at vertex v, attempt every colour 1..m; isSafeColor checks that no neighbour of v (per the adjacency matrix) already has that colour. if a colour is safe, assign it and recurse to vertex v+1; if the rest of the graph cannot be coloured with m colours, undo the assignment and try the next colour at v.

approach_2 (finding the chromatic number): run the above backtracking colourer for m = 1, 2, 3, ... in increasing order and stop at the first m for which a complete valid colouring is found. that m is the chromatic number of the graph, since any smaller m was already shown to be infeasible by the backtracking search.

verification: after a colouring is produced, verifyColoring scans every pair of adjacent vertices (graph[u][v] == 1) and confirms colors[u] != colors[v] for all of them, independently confirming the backtracking result is valid.

Time and Space Complexity:

time complexity: O(m^V) in the worst case for a fixed m (V vertices, up to m choices per vertex), and since the chromatic number is found by retrying m = 1, 2, ..., chromatic_number, the overall cost is dominated by the final successful (or the largest attempted) value of m.

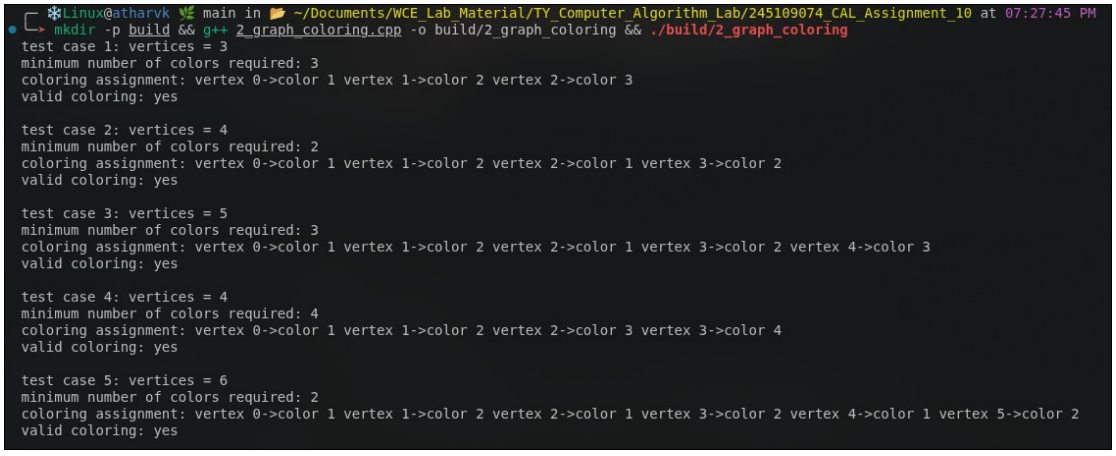
space complexity: O(V) for the colour array and O(V^2) for the adjacency matrix.

Code:

```
#include<iostream>
#include<vector>
using namespace std;
bool isSafeColor(vector<vector<int>>&graph,vector<int>&colors,int v,int c){
   int n=graph.size();
   for(int u=0;u<n;u++){
      if(graph[v][u]&&colors[u]==c)
         return false;
   }
   return true;
}
bool colorGraph(vector<vector<int>>&graph,int m,vector<int>&colors,int v){
   int n=graph.size();
   if(v==n)
      return true;
   for(int c=1;c<=m;c++){
      if(isSafeColor(graph,colors,v,c)){
         colors[v]=c;
         if(colorGraph(graph,m,colors,v+1))
            return true;
         colors[v]=0;
      }
   }
   return false;
}
bool verifyColoring(vector<vector<int>>&graph,vector<int>&colors){
   int n=graph.size();
   for(int u=0;u<n;u++)
      for(int v=0;v<n;v++)
```

```
            if(graph[u][v]&&colors[u]==colors[v])
                return false;
    return true;
}
void runTestCase(int tc,vector<vector<int>>graph){
    int n=graph.size();
    vector<int>colors(n,0);
    int chromaticNumber=n;
    for(int m=1;m<=n;m++){
        fill(colors.begin(),colors.end(),0);
        if(colorGraph(graph,m,colors,0)){
            chromaticNumber=m;
            break;
        }
    }
    cout<<"test case "<<tc<<": vertices = "<<n<<endl;
    cout<<"minimum number of colors required: "<<chromaticNumber<<endl;
    cout<<"coloring assignment: ";
    for(int i=0;i<n;i++)
        cout<<"vertex "<<i<<"->color "<<colors[i]<<" ";
    cout<<endl;
    cout<<"valid coloring: "<<(verifyColoring(graph,colors)?"yes":"no")<<endl;
    cout<<endl;
}
int main(){
    vector<vector<int>>g1={{0,1,1},{1,0,1},{1,1,0}};
    vector<vector<int>>g2={{0,1,0,1},{1,0,1,0},{0,1,0,1},{1,0,1,0}};
    vector<vector<int>>g3={{0,1,0,0,1},{1,0,1,0,0},{0,1,0,1,0},{0,0,1,0,1},{1,0,0,1,0}};
    vector<vector<int>>g4={{0,1,1,1},{1,0,1,1},{1,1,0,1},{1,1,1,0}};
    vector<vector<int>>g5={{0,1,0,0,0,1},{1,0,1,0,0,0},{0,1,0,1,0,0},{0,0,1,0,1,0},{0,0,0,1,0,1},{1,0,0,0,1,0}};
    runTestCase(1,g1);
    runTestCase(2,g2);
    runTestCase(3,g3);
    runTestCase(4,g4);
    runTestCase(5,g5);
    return 0;
}
```

Output:



Q3. Hamiltonian Cycle (Backtracking)

Problem Statement:

In logistics and delivery networks, a courier must visit every depot exactly once and return to the starting point without repeating any location - this mirrors the Hamiltonian Cycle problem. Implement the Hamiltonian cycle problem using backtracking given a graph adjacency matrix, display the cycle if one exists or report that none exists, and analyze the time complexity while discussing why it belongs to the NP-complete class.

Approach:

approach_1: fix vertex 0 as the start of the path (any Hamiltonian cycle can be rotated to start there). build the path one vertex at a time; at each step try every unvisited vertex v, and isSafeVertex checks both that an edge exists from the previous vertex in the path to v and that v has not already been used. if v is safe, add it to the path and recurse; if continuing from v cannot complete a full cycle, remove it (backtrack) and try the next candidate.

approach_2 (closing the cycle): once the path contains all n vertices (pos == n), the search does not simply stop - it additionally checks that an edge exists from the last vertex back to the starting vertex (path[0]). only if that closing edge exists is the path reported as a true Hamiltonian cycle; otherwise that branch also fails and backtracking continues.

Time and Space Complexity:

time complexity: O(n!) in the worst case, since in the worst case backtracking tries all permutations of the remaining n-1 vertices before finding a cycle or exhausting the search; adjacency checks make each step O(1) with an adjacency matrix.

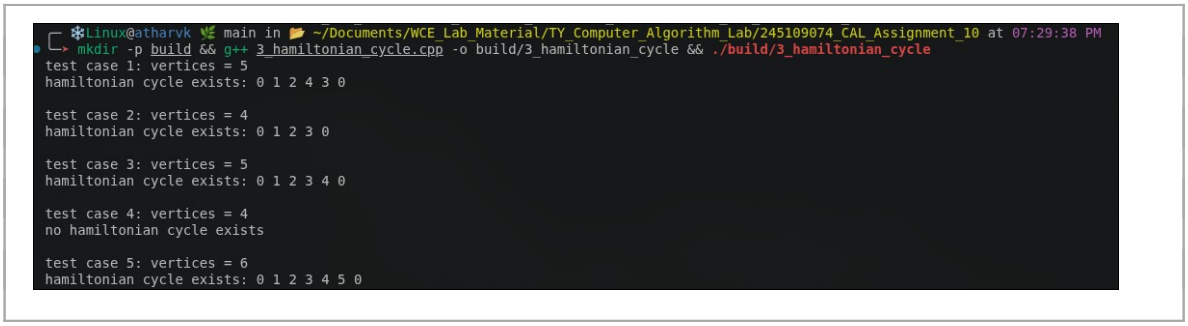
space complexity: O(n) for the path array and the recursion stack, plus O(n^2) for the adjacency matrix. why it is NP-complete: a proposed cycle can be verified in O(n) time by simply checking that consecutive vertices (including the wraparound edge) are all connected and that every vertex appears exactly once, so the problem is in NP; it is also NP-hard because it generalises problems such as the travelling salesman decision problem, and no polynomial-time algorithm is known for solving it on arbitrary graphs, which places it in NP-complete.

Code:

```
#include<iostream>
#include<vector>
using namespace std;
bool isSafeVertex(vector<vector<int>>&graph,vector<int>&path,int v,int pos){
   if(!graph[path[pos-1]][v])
      return false;
   for(int i=0;i<pos;i++)
      if(path[i]==v)
         return false;
   return true;
}
bool hamCycleUtil(vector<vector<int>>&graph,vector<int>&path,int pos){
   int n=graph.size();
   if(pos==n)
      return graph[path[pos-1]][path[0]]==1;
   for(int v=1;v<n;v++){
      if(isSafeVertex(graph,path,v,pos)){
         path[pos]=v;
         if(hamCycleUtil(graph,path,pos+1))
            return true;
         path[pos]=-1;
      }
   }
   return false;
}
void runTestCase(int tc,vector<vector<int>>graph){
```

```
    int n=graph.size();
    vector<int>path(n,-1);
    path[0]=0;
    cout<<"test case "<<tc<<": vertices = "<<n<<endl;
    if(hamCycleUtil(graph,path,1)){
        cout<<"hamiltonian cycle exists: ";
        for(int v:path)
            cout<<v<<" ";
        cout<<path[0]<<endl;
    }
    else
        cout<<"no hamiltonian cycle exists"<<endl;
    cout<<endl;
}
int main(){
    vector<vector<int>>g1={{0,1,0,1,0},{1,0,1,1,1},{0,1,0,0,1},{1,1,0,0,1},{0,1,1,1,0}};
    vector<vector<int>>g2={{0,1,0,1},{1,0,1,0},{0,1,0,1},{1,0,1,0}};
    vector<vector<int>>g3={{0,1,1,1,1},{1,0,1,1,1},{1,1,0,1,1},{1,1,1,0,1},{1,1,1,1,0}};
    vector<vector<int>>g4={{0,1,0,0},{1,0,1,0},{0,1,0,1},{0,0,1,0}};
    vector<vector<int>>g5={{0,1,0,0,0,1},{1,0,1,0,0,0},{0,1,0,1,0,0},{0,0,1,0,1,0},{0,0,0,1,0,1},{1,0,0,0,1,0}};
    runTestCase(1,g1);
    runTestCase(2,g2);
    runTestCase(3,g3);
    runTestCase(4,g4);
    runTestCase(5,g5);
    return 0;
}
```

Output:



Q4. Approximation Algorithm for Vertex Cover

Problem Statement:

In network security, placing monitoring devices on the minimum number of routers such that every communication link is observed by at least one device mimics the Vertex Cover problem. Implement an approximation algorithm for vertex cover given a graph (adjacency list of edges), output the approximate vertex cover set and its size, compare it against the optimal solution for small graphs, and derive/discuss the algorithm's approximation ratio (2-approximation).

Approach:

approach_1 (2-approximation, the implemented solution): repeatedly pick any edge (u, v) that is not yet covered, add both endpoints u and v to the cover set, and mark every edge incident to u or v as covered (since both endpoints are now in the cover, all their edges are automatically satisfied). repeat until every edge is covered. this is the classical approximation algorithm and is simulated directly: a boolean array tracks which edges are already covered, and picking an uncovered edge's both endpoints is equivalent to the standard maximal-matching-based 2-approximation.

approach_2 (optimal, used only for comparison on small graphs): brute-force every subset size from 0 upwards, and for each size enumerate all C(n, size) subsets of vertices (via bitmasks), checking with isCover whether that subset touches every edge. the first size for which some subset is a valid cover is the true minimum vertex cover size. this is exponential (O(2^n)) and is only used here because the test graphs are small enough to make brute force feasible; it is not part of the approximation algorithm itself.

Time and Space Complexity:

time complexity: O(E) for the approximation algorithm (each edge is scanned once to pick it, and marking covered edges touches each edge a bounded number of times), versus O(2^n x E) for the brute-force optimal solution used only for small-graph comparison.

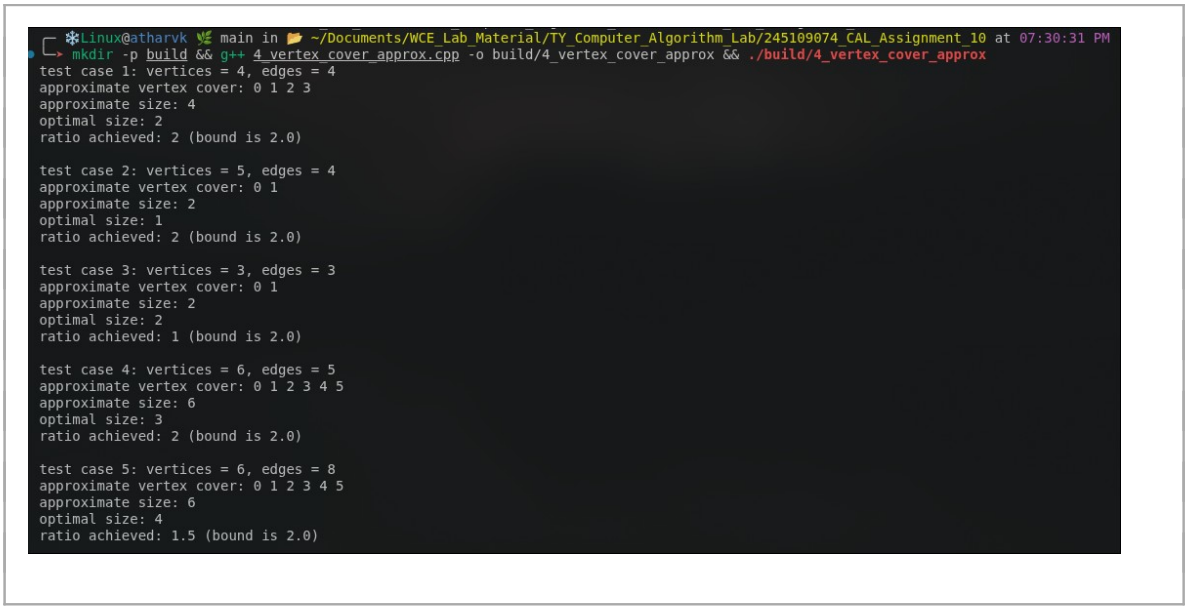
space complexity: O(V) for the cover set, O(E) for the covered-edges tracking array, and O(2^n) implicitly explored (not stored) during the brute-force search.

Code:

```
#include<iostream>
#include<vector>
#include<set>
using namespace std;
set<int>approxVertexCover(int n,vector<pair<int,int>>edges){
   vector<bool>covered(edges.size(),false);
   set<int>cover;
   for(int i=0;i<(int)edges.size();i++){
      if(!covered[i]){
         int u=edges[i].first;
         int v=edges[i].second;
         cover.insert(u);
         cover.insert(v);
         for(int j=0;j<(int)edges.size();j++){
            if(edges[j].first==u||edges[j].second==u||edges[j].first==v||edges[j].second==v)
               covered[j]=true;
         }
      }
   }
   return cover;
}
bool isCover(int n,vector<pair<int,int>>&edges,vector<int>&subset){
   set<int>s(subset.begin(),subset.end());
   for(auto&e:edges)
      if(!s.count(e.first)&&!s.count(e.second))
         return false;
```

```
    return true;
}
int optimalVertexCover(int n,vector<pair<int,int>>edges){
    for(int size=0;size<=n;size++){
        vector<int>subset;
        for(int mask=0;mask<(1<<n);mask++){
            if(__builtin_popcount(mask)!=size)
                continue;
            subset.clear();
            for(int i=0;i<n;i++)
                if(mask&(1<<i))
                    subset.push_back(i);
            if(isCover(n,edges,subset))
                return size;
        }
    }
    return n;
}
void runTestCase(int tc,int n,vector<pair<int,int>>edges){
    set<int>approx=approxVertexCover(n,edges);
    int optimal=optimalVertexCover(n,edges);
    cout<<"test case "<<tc<<": vertices = "<<n<<", edges = "<<edges.size()<<endl;
    cout<<"approximate vertex cover: ";
    for(int v:approx)
        cout<<v<<" ";
    cout<<endl;
    cout<<"approximate size: "<<approx.size()<<endl;
    cout<<"optimal size: "<<optimal<<endl;
    cout<<"ratio achieved: "<<(double)approx.size()/optimal<<" (bound is 2.0)"<<endl;
    cout<<endl;
}
int main(){
    vector<pair<int,int>>e1={{0,1},{1,2},{2,3},{3,0}};
    vector<pair<int,int>>e2={{0,1},{0,2},{0,3},{0,4}};
    vector<pair<int,int>>e3={{0,1},{1,2},{2,0}};
    vector<pair<int,int>>e4={{0,1},{1,2},{2,3},{3,4},{4,5}};
    vector<pair<int,int>>e5={{0,1},{0,2},{1,2},{1,3},{2,4},{3,4},{3,5},{4,5}};
    runTestCase(1,4,e1);
    runTestCase(2,5,e2);
   runTestCase(3,3,e3);
    runTestCase(4,6,e4);
    runTestCase(5,6,e5);
    return 0;
}
```

Output:


❄Linux@atharvk 🌿 main in 📂 ~/Documents/WCE_Lab_Material/TY_Computer_Algorithm_Lab/245109074_CAL_Assignment_10 at 07:30:31 PM
mkdir -p build && g++ 4_vertex_cover_approx.cpp -o build/4_vertex_cover_approx && ./build/4_vertex_cover_approx
test case 1: vertices = 4, edges = 4
approximate vertex cover: 0 1 2 3
approximate size: 4
optimal size: 2
ratio achieved: 2 (bound is 2.0)
test case 2: vertices = 5, edges = 4
approximate vertex cover: 0 1
approximate size: 2
optimal size: 1
ratio achieved: 2 (bound is 2.0)
test case 3: vertices = 3, edges = 3
approximate vertex cover: 0 1
approximate size: 2
optimal size: 2
ratio achieved: 1 (bound is 2.0)
test case 4: vertices = 6, edges = 5
approximate vertex cover: 0 1 2 3 4 5
approximate size: 6
optimal size: 3
ratio achieved: 2 (bound is 2.0)
test case 5: vertices = 6, edges = 8
approximate vertex cover: 0 1 2 3 4 5
approximate size: 6
optimal size: 4
ratio achieved: 1.5 (bound is 2.0)